arXiv:2608.19842v1 [cs.AI] 20 Aug 2026

SAPO: Single-Rollout Autoregressive Policy Optimization for Agentic Reinforcement Learning

Dayang Liang[1*] **Lang Feng**[2*] **Bo An**[2] **Yunlong Liu**[1†]
[1]Xiamen University, China
[2]Nanyang Technological University, Singapore
dyliang@stu.xmu.edu.cn, lang005@e.ntu.edu.sg, ylliu@xmu.edu.cn,

Abstract

Agentic reinforcement learning (RL) has become a critical stage in the post-training of large language models. Existing critic-free, group-relative methods estimate policy advantages from multiple rollouts, avoiding the substantial memory overhead of conventional proximal policy optimization (PPO) and achieving strong performance on long-horizon interactive tasks. Despite their success, recent studies revealed three limitations: (1) Lack explicit value generalization and effective temporal credit assignment; (2) Suffer from potential advantage collapse in long-horizon complex tasks; (3) Require a costly trade-off between sampling budget and policy performance. In this work, we propose **S**ingle-rollout **A**utoregressive **P**olicy **O**ptimization (**SAPO**), a low-memory and compute-efficient framework in which the policy and value functions share a single autoregressive backbone. SAPO exploits the autoregressive structure of LLMs to produce policy and value predictions at distinct causal boundaries with shared parameters, while independently optimizing the PPO objectives and auxiliary on-policy SARSA objectives. To robustly estimate the contribution of each turn, we further introduce a trajectory-level generalized advantage estimator that combines λ-returns with batch normalization. Experiments across ALFWorld and WebShop with Qwen2.5-1.5B/7B show that SAPO trains stably and outperforms PPO and GRPO by mean **+15.1** and **+12.1** percentage points, respectively, while eliminating the memory cost of a separate critic model and reducing per-iteration runtime by **33.2%** over PPO. (Going On)

1 Introduction

Recent advances in agentic reinforcement learning (RL) have transformed large language models from passive text generators into agents capable of long-horizon interaction such as reasoning, tool use, web search, and code execution tasks. Large-scale RL post-training has produced substantial improvements in mathematical reasoning and coding [17, 5, 7], while interactive systems such as Search-R1 demonstrate that policies can learn to acquire external information through multi-turn actions [8]. These successes suggest that learning directly from environmental feedback is becoming a critical pathway toward increasingly autonomous and general-purpose language agents.

Most agentic RL approaches build on proximal policy optimization (PPO) or its critic-free, group-relative variants. Conventional PPO provides token- or turn-level credit assignment through a learned value function and generalized advantage estimation (GAE), but typically requires a policy-scale critic with substantial memory and computation budgets [13, 21]. GRPO avoids this critic by normalizing rewards across multiple sampled for the same task [17]. Nevertheless, recent studies expose three limitations. First, when all rollouts in a group receive identical or nearly identical

[*]Equal Contribution
[†]Corresponding author

Preprint. Under review.

rewards—an increasingly common event under sparse feedback and long-horizon failures—the normalized advantages vanish, causing zero-gradient or advantage-collapse regions [29]. Second, assigning a trajectory-level group advantage uniformly to all generated tokens provides weak temporal credit assignment and cannot generalize value estimates across states, prompts, or policy iterations; learned critics can identify low-value repetitive or erroneous prefixes that group-relative estimators miss [30, 31, 7]. Third, group-relative estimation entails an unfavorable compute–quality trade-off: small groups yield noisy baselines, whereas larger groups improve stability only by multiplying rollout cost and imposing synchronization barriers in long, variable-length agent trajectories [3, 6].

Several work attempts to mitigate these problems. DAPO improves group-relative learning through dynamic sampling, asymmetric clipping, and token-level policy losses, but remains dependent on multiple rollouts and group statistics [29]. ReMax and RLOO eliminate the critic using greedy or leave-one-out baselines, while VinePPO estimates intermediate values through Monte Carlo continuations; these methods reduce training-state memory but retain coarse credit assignment or introduce additional generation cost [12, 2, 9]. Conversely, Open-Reasoner-Zero, VC-PPO, and VAPO restore explicit value models and demonstrate that a well-trained critic can substantially improve long-horizon reasoning, particularly through critic pretraining and improved GAE [7, 30, 31]. However, these methods maintain a separate, often policy-scale value network. Hydra-PPO partially shares a frozen backbone, but still requires role-specific adapters and heads, and its fully shared variant reveals potential policy–value interference [15]. POISE reuses actor hidden states through a lightweight probe but requires cross-rollout estimation [3], whereas SAO enables asynchronous single-rollout training while relying on a separately pretrained critic with more frequent value updates [6]. Thus, efficient single-rollout learning with explicit temporal value estimation remains unresolved.

To address this, we propose Single-Rollout Autoregressive Policy Optimization (SAPO), a lightweight RL framework that preserves value-based learning without maintaining a separate critic or relying on multiple rollouts. SAPO builds on a simple observation: the information flow required by actor–critic learning naturally follows the causal order of language generation. A value estimate should summarize the state before an action is taken, whereas evaluating that action may additionally condition on what has been generated. This alignment allows policy generation and value estimation to be expressed at appropriate positions within one autoregressive stream, with causal masking enforcing their distinct conditioning contexts. Consequently, the same backbone can generate actions and learn generalized value estimates while reusing its parameters and intermediate computation. SAPO thus reconciles two competing goals in existing approaches: it retains the explicit value generalization and temporal credit assignment of actor–critic algorithms, yet approaches the memory and computational efficiency sought by critic-free, group-relative optimization. Its central contribution is therefore not weight sharing alone, but a correspondence between autoregressive modeling and actor–critic learning that enables efficient value-based optimization within a single causal model.

We evaluate SAPO on ALFWorld and WebShop using Qwen2.5-1.5B and Qwen2.5-7B backbone models. SAPO trains stably from one rollout per prompt and improves task success over conventional PPO and GRPO by **+15.1** and **+12.1** percentage points, respectively. It simultaneously eliminates the memory footprint of a separate critic and reduces per-iteration runtime by **33.2%** relative to PPO. Our contributions are threefold.

- We introduce a single-rollout autoregressive actor–critic architecture that jointly represents and trains policy and value functions through causal-boundary readouts.
- We develop a trajectory-level advantage estimator that combines λ-returns with batch normalization, enabling explicit turn-level credit assignment without group-relative sampling.
- We demonstrate on long-horizon agentic tasks that this unified framework achieves a favorable combination of performance, training stability, memory efficiency, and runtime efficiency.

2 Related Work

Agentic RL extends reinforcement learning with verifiable rewards from single-turn response optimization to multi-turn decision making, enabling LLM agents to learn from the consequences of actions taken in external environments. A prominent line of work builds on critic-free, group-relative optimization. GRPO [17] replaces a learned value baseline with within-prompt outcome comparison,

a formulation used by DeepSeek-R1 [5] and refined by DAPO [29] with dynamic sampling and asymmetric clipping, GSPO [33] with sequence-level importance ratios, and GVPO [32] through variance-aware gradient weighting. This paradigm has been extended to interactive settings: Search-R1 [8] and DeepResearcher [34] optimize iterative retrieval, ToRL [11] and WebSailor [10] train tool use and web navigation, and RAGEN [25] characterizes optimization instabilities in multi-turn training. Long-horizon tasks have also motivated more structured credit mechanisms. ARChER [35] and HiPER [14] decompose planning and execution across temporal scales, AgentGym-RL [26] progressively increases the interaction horizon, and IGPO [23] derives dense intrinsic rewards from information gain during search.

Efficiency-oriented LLM RL has developed along critic-free advantage estimation, parameter sharing between policy and value functions, and systems-level execution. ReMax [12] and RLOO [2] replace learned values with greedy or leave-one-out baselines, while GRPO [17] normalizes rewards within each prompt group. Eliminating the value network reduces parameters and optimizer state, but these estimators require additional decoded trajectories and provide no value generalization across states. VinePPO [9] obtains intermediate value estimates from branched Monte Carlo continuations, likewise exchanging critic training for increased rollout computation. Parameter-sharing approaches retain explicit value learning while reducing model duplication. Hydra-PPO [15] places policy and value adapters over a frozen backbone; J-Hydra [15] further shares the trainable adapter, with potential interference between the two objectives. POISE [3] predicts returns from actor hidden states using a lightweight probe, but relies on cross-rollout construction to decouple value estimation from the evaluated trajectory. SAO [6] addresses rollout underutilization through asynchronous single-rollout collection, while retaining a separately pretrained critic with more frequent value updates. HybridFlow [18] instead improves distributed execution through model placement, resharding, and scheduling, independently of the advantage estimator. SAPO is distinguished by jointly representing the policy, state value, and action value in one causal autoregressive model, thereby eliminating both group sampling and the separate critic backbone while retaining explicit temporal credit assignment.

3 Preliminaries

Multi-Turn Agentic RL. We consider RL for LLM-based agents that interact with an environment over multiple turns, where the interaction process is formulated as a finite-horizon Markov Decision Process (MDP). Given a task instance $\boldsymbol{x} \in p(X)$, at each turn $t = 1, 2, \ldots, T$, an agent policy π_θ observes a state $\boldsymbol{s}_t \in \mathcal{S}$ and generates a textual action $\boldsymbol{a}_t \in \mathcal{A}$, and then transitions to the next state $\boldsymbol{s}_{t+1} \in \mathcal{S}$ while yielding a scalar reward $r_t \in \mathbb{R}$. The interaction unfolds as a trajectory $\boldsymbol{\tau} = \{(\boldsymbol{s}_1, \boldsymbol{a}_1, r_1), (\boldsymbol{s}_2, \boldsymbol{a}_2, r_2), ..., (\boldsymbol{s}_T, \boldsymbol{a}_T, r_T)\}$, where T denotes the trajectory length in interaction turns. In practical tasks such as ALFWorld, the agent generates a response as its action for each observation. This response is structured within `<think></think>` and `<action></action>` tags, where the former contains the reasoning process, while the latter specifies the actual action executed in the environment. Notably, for most task settings, reward signals are sparse and delayed, e.g., the environment provides an outcome reward $R(\boldsymbol{\tau})$ only after the trajectory terminates.

3.1 Group-relative Policy Optimization

Recent agentic RL methods for LLMs commonly adopt a group-relative policy optimization paradigm. Given a task instance $\boldsymbol{x}$, the old policy $\pi_{\theta_{\text{old}}}$ samples a group of N candidate trajectories $\mathcal{G}_x = \{\boldsymbol{\tau}_1, \boldsymbol{\tau}_2, \ldots, \boldsymbol{\tau}_N\}$, where each trajectory corresponds to one complete rollout. Each trajectory $\boldsymbol{\tau}_i$ receives a scalar reward $R(\boldsymbol{\tau}_i)$ that reflects the overall quality or success of the generated outcome. Instead of learning an advantage function $A(\boldsymbol{s}_t, \boldsymbol{a}_t)$ with critic networks in PPO [16], group-based RL computes the advantage using only statistics within the sampled group:

$$A(\boldsymbol{\tau}_i) = \texttt{GroupNormalization}\left(\{R(\boldsymbol{\tau}_i)\}_{i=1}^N\right).$$

In GRPO [17], the advantage is evaluated by normalizing each trajectory reward with the mean and variance of group rewards $(\{R(\boldsymbol{\tau}_i)\}_{i=1}^N)$. This sampling-based estimator reduces the memory and computational overhead introduced by the critic architecture in conventional PPO.

3.2 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is an actor-critic RL algorithm previously used for LLM post-training. Given prompts sampled from a dataset, the policy generates response trajectories and receives rewards from task-specific environments or learned reward models. PPO updates the policy by maximizing a clipped surrogate objective,

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t\Big[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big],$$

where $\rho_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is the likelihood ratio and $\hat{A}_t$ is an advantage estimate. The clipping operation constrains abrupt policy changes and improves optimization stability. A learned critic $V_\phi(s_t)$ predicts expected returns, enabling advantage estimation—commonly through generalized advantage estimation—and propagating delayed rewards to earlier decisions. In LLMs, states correspond to token prefixes or interaction histories, while actions are generated tokens. Although effective for long-horizon credit assignment, PPO typically requires separate policy, critic, reference, and rollout models, resulting in substantial memory and computational overhead.

4 Method

SAPO is a single-rollout actor–critic method in which one causal language model represents the policy, the state value, and the action value. As shown in Figure 1, the design follows the temporal structure of an agent turn: the model first summarizes the current interaction state, then generates a textual action, and finally evaluates that action before observing its environmental consequence. This order matches the conditioning structure required by actor–critic learning and allows all three quantities to be extracted from one autoregressive sequence.

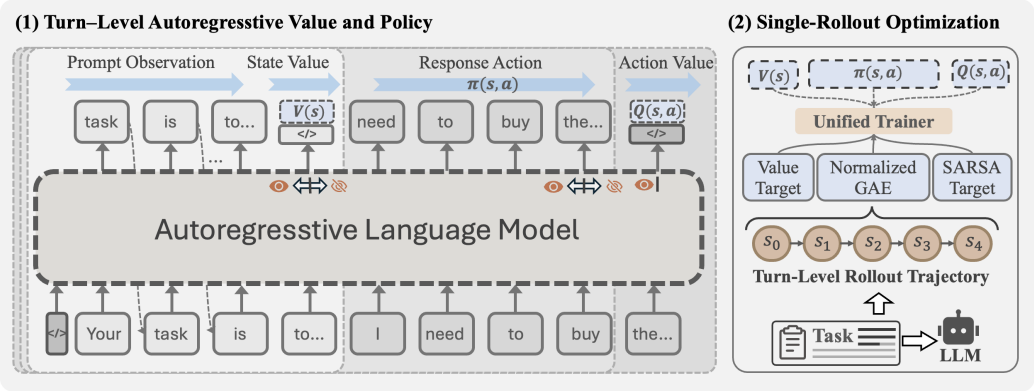

Figure 1: Framework of SAPO.

We begin with introducing a two-token value basis that reads bounded scalar values from the language-model logits at two causal boundaries (Section 4.1). Second, we construct turn-level advantages and value targets by traversing each sampled trajectory backward (Section 4.2). Third, we jointly optimize the token-level policy objective and the turn-level value objectives in one actor update (Section 4.3). Algorithm 1 summarizes the complete procedure.

4.1 Autoregressive Actor–Critic Representation

Causal factorization of an agent turn. At turn t, let $\boldsymbol{c}_t^s$ denote the tokenized context representing state $\boldsymbol{s}_t$, and let $\boldsymbol{a}_t = (a_{t,1}, \ldots, a_{t,M_t})$ be the generated response. We write $\boldsymbol{c}_t^{sa} = [\boldsymbol{c}_t^s; \boldsymbol{a}_t]$ for their concatenation. A valid state value may depend on $\boldsymbol{c}_t^s$ but not on the yet-unseen action, whereas an action value may additionally depend on the complete $\boldsymbol{a}_t$ but not on the subsequent reward or observation. These dependencies occur naturally at the two boundaries of causal decoding: immediately before the first response token and immediately after the last response token. SAPO reads $V(\boldsymbol{s}_t)$ and $Q(\boldsymbol{s}_t, \boldsymbol{a}_t)$ at these respective boundaries while generating $\boldsymbol{a}_t$ between them. The

causal mask enforces the desired information separation without separate encoders or manually detached context representations.

A shared two-token value basis. Let $\boldsymbol{z}_\theta(\boldsymbol{c}) \in \mathbb{R}^{|\mathcal{W}|}$ be the next-token logits produced by the causal language model after context $\boldsymbol{c}$, where $\mathcal{W}$ is its vocabulary. We reserve two existing vocabulary entries, w^+ and w^-, exclusively as a value basis. For any causal context, their relative evidence defines a normalized scalar readout:

$$p_\theta(\boldsymbol{c}) = \text{clip}\left(\left(\frac{z_{\theta,w^+}(\boldsymbol{c}) - z_{\theta,w^-}(\boldsymbol{c})}{\tau_v}\right), -1, 1\right) \tag{1}$$

where $\tau_v > 0$ is a value temperature. For tasks whose discounted returns lie in $[-R_{\max}, R_{\max}]$, SAPO parameterizes:

$$V_\theta(\boldsymbol{s}_t) = R_{\max} p_\theta(\boldsymbol{c}_t^s), \qquad Q_\theta(\boldsymbol{s}_t, \boldsymbol{a}_t) = R_{\max} p_\theta(\boldsymbol{c}_t^{sa}). \tag{2}$$

The logit difference makes the readout invariant to a common shift of the two logits, while the clipping supplies a learning range matched to the return scale. Importantly, w^+ and w^- are readouts rather than generated reasoning tokens: neither token is sampled, appended to the context, or revealed to the environment.

Separation from the action distribution. The reserved entries must not become artificial actions. We therefore define the policy vocabulary as $\mathcal{W}_{\text{act}} = \mathcal{W} \setminus \{w^+, w^-\}$ and normalize action probabilities only over this set:

$$\pi_\theta(a \mid \boldsymbol{c}) = \frac{\exp z_{\theta,a}(\boldsymbol{c})}{\sum_{w \in \mathcal{W}_{\text{act}}} \exp z_{\theta,w}(\boldsymbol{c})}, \qquad a \in \mathcal{W}_{\text{act}}. \tag{3}$$

The same restriction is applied during rollout, policy-log-probability evaluation, and entropy computation. Thus, policy learning never rewards or suppresses the value basis through the action softmax, while the raw logits of the two reserved entries remain available to the value objectives. Policy and value learning share the transformer and language-model head, but retain distinct semantics and supervision.

Single-stream evaluation. For every sampled response, PPO already evaluates the actor on $[\boldsymbol{c}_t^s; \boldsymbol{a}_t]$ to obtain old token log probabilities. SAPO extends this actor evaluation by gathering the two reserved logits at the final state-context position and the final valid action position. The former yields $V_{\theta_{\text{old}}}(\boldsymbol{s}_t)$ and the latter yields $Q_{\theta_{\text{old}}}(\boldsymbol{s}_t, \boldsymbol{a}_t)$; no second model or repeated encoding of the state–action prefix is required. During optimization, the same actor forward simultaneously produces current policy log probabilities, state values, and action values. Hence, the precise computational benefit of SAPO is the removal of the independent critic forward/backward path and its training state, rather than an assumption that value estimates are obtained without any actor evaluation.

4.2 Single-Rollout Trajectory Advantage Estimation

Group-relative algorithms estimate advantages by comparing several trajectories sampled for the same task. SAPO instead samples one trajectory per task and transfers delayed feedback across its interaction turns using the learned state value. All targets are computed from rewards observed in the environment and values produced by the frozen rollout policy $\pi_{\theta_{\text{old}}}$; current value predictions are never used to construct their own targets.

Trajectory-wise generalized advantage estimation. For a trajectory $\boldsymbol{\tau} = \{(\boldsymbol{s}_t, \boldsymbol{a}_t, r_t, d_t)\}_{t=1}^T$, let d_t indicate termination after turn t and let $V_t^{\text{old}} = V_{\theta_{\text{old}}}(\boldsymbol{s}_t)$. We compute the temporal-difference residual and the generalized advantage recursively as:

$$\delta_t = r_t + \gamma(1 - d_t) V_{t+1}^{\text{old}} - V_t^{\text{old}}, \tag{4}$$

$$A_t^{\text{GAE}} = \delta_t + \gamma\lambda(1 - d_t) A_{t+1}^{\text{GAE}}, \tag{5}$$

with $A_{T+1}^{\text{GAE}} = 0$. A trajectory truncated at the environment horizon is treated as a finite-horizon terminal trajectory, so its final bootstrap value is zero. This recursion gives different turns different

learning signals even when the environment supplies only a terminal outcome, while λ controls the usual bias–variance trade-off.

The same backward pass provides complementary targets for the two causal value boundaries. The state value is trained toward the λ-return, whereas the action value is trained toward a on-policy SARSA target:

$$y_t^V = V_t^{\text{old}} + A_t^{\text{GAE}}, \qquad y_t^Q = r_t + \gamma(1-d_t)Q_{t+1}^{\text{old}}. \tag{6}$$

The first target propagates long-range outcome information to the pre-action state boundary. The second grounds the post-action boundary in the immediate consequence of the selected response. We deliberately construct the policy advantage from GAE rather than from the difference between simultaneously learned Q and V estimates: this prevents early action-value miscalibration from directly perturbing the policy while still using the Q objective to train an action-conditioned value representation.

Batch-normalized turn advantages. Before assigning advantages to response tokens, we normalize them over the valid turns in the current training batch. If an environment exposes an invalid-action indicator m_t^{inv}, we first form $\widetilde{A}_t = A_t^{\text{GAE}} - c_{\text{inv}} m_t^{\text{inv}}$; otherwise $c_{\text{inv}} = 0$. With batch statistics $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}$, the policy advantage is

$$\widehat{A}_t = \frac{\widetilde{A}_t - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}} + \epsilon_{\text{adv}}}. \tag{7}$$

Only after this turn-level normalization do we broadcast $\widehat{A}_t$ to all valid tokens in $\boldsymbol{a}_t$. Normalizing before broadcasting prevents long responses from receiving disproportionate weight in the advantage statistics. Unlike group normalization, Equation 7 compares learning signals across independent turns and tasks; it therefore does not require multiple synchronized rollouts of the same prompt.

4.3 Joint Policy and Value Optimization

Token-level policy objective. Although the environment acts at the turn level, each textual action contains multiple autoregressive decisions. Let $\ell_{t,j}^{\text{old}}$ and $\ell_{t,j}(\theta)$ be the old and current log probabilities of token $a_{t,j}$ under the restricted action distribution in Equation 3. The importance ratio is

$$\rho_{t,j}(\theta) = \exp\big(\ell_{t,j}(\theta) - \ell_{t,j}^{\text{old}}\big). \tag{8}$$

Every valid response token in turn t shares $\widehat{A}_t$, yielding the clipped policy loss

$$\mathcal{L}_{\text{pol}}(\theta) = -\mathbb{E}_{t,j}\Big[\min\Big(\rho_{t,j}(\theta)\widehat{A}_t, \text{clip}(\rho_{t,j}(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\Big)\Big], \tag{9}$$

where the expectation includes only valid action tokens. This preserves PPO's token-level trust-region surrogate while replacing both the separate critic and group-relative advantage estimator.

Clipped value objectives. We optimize the value readouts in their normalized probability space. Define $p_t^V = V_\theta(\boldsymbol{s}_t)/R_{\max}$ and $p_t^Q = Q_\theta(\boldsymbol{s}_t, \boldsymbol{a}_t)/R_{\max}$, with frozen rollout-time predictions $p_t^{V,\text{old}}$ and $p_t^{Q,\text{old}}$. Accordingly, the normalized targets are $\bar{y}_t^V = y_t^V/R_{\max}$ and $\bar{y}_t^Q = y_t^Q/R_{\max}$ with normalized range $[-1, 1]$. For $X \in \{V, Q\}$, let

$$p_t^{X,\text{clip}} = p_t^{X,\text{old}} + \text{clip}\Big(p_t^X - p_t^{X,\text{old}}, -\epsilon_X, \epsilon_X\Big). \tag{10}$$

We use the PPO-style clipped regression loss

$$\mathcal{L}_X(\theta) = \frac{1}{2}\mathbb{E}_t\Big[\max\Big((p_t^X - \bar{y}_t^X)^2, (p_t^{X,\text{clip}} - \bar{y}_t^X)^2\Big)\Big], \qquad X \in \{V, Q\}. \tag{11}$$

Both losses are averaged over valid turns, not over response tokens. This gives each environment decision equal weight regardless of response length.

Unified objective. The final loss combines policy optimization, the two causal value objectives, reference-policy regularization, and entropy regularization:

$$\mathcal{L}_{SAPO}(\theta) = \mathcal{L}_{\text{pol}} + c_V\mathcal{L}_V + c_Q\mathcal{L}_Q + \beta\mathcal{L}_{\text{KL}} - c_H\mathcal{H}(\pi_\theta). \tag{12}$$

The policy, state-value, and action-value terms use separate masks but update the same transformer and language-model head in one backward pass. Coefficients c_V and c_Q control interference between generation and value learning, while the KL and entropy terms regularize policy drift and exploration. This objective retains distinct supervision for the three roles without allocating role-specific model parameters.

Algorithm 1 Single-Rollout Autoregressive Policy Optimization

Require: Policy π_θ, reference policy π_{ref}, task batch $\mathcal{B}$, γ, λ

1: **for** each training iteration **do**
2: Set $\theta_{\text{old}} \leftarrow \theta$ and sample one trajectory per task with $\pi_{\theta_{\text{old}}}$
3: Evaluate sampled state–action sequences with the old actor to obtain token log probabilities, V^{old}, and Q^{old}
4: Traverse each trajectory backward using Equations 4–6
5: Normalize turn-level policy advantages using Equation 7 and broadcast them to valid action tokens
6: Run one joint actor forward on the sampled sequences to obtain current policy, V, and Q predictions
7: Update θ by minimizing Equation 12
8: **end for**

In implementation, trajectory identifiers and turn indices are retained until after the backward target computation, ensuring that batching, duplication, or shuffling cannot break temporal adjacency. Reward is used only as a target after environment execution and is never included in the action-value input. Together with the causal boundary construction, this preserves the semantics of both state and action values throughout single-stream training.

Type	Method	ALFWorld							WebShop	
		Pick	Look	Clean	Heat	Cool	Pick2	All	Score	Succ.
Closed-Source Model										
Prompting	GPT-4o	75.3	60.8	31.2	56.7	21.6	49.8	48.0	31.8	23.7
Prompting	Gemini-2.5-Pro	92.8	63.3	62.1	69.0	26.6	58.7	60.3	42.5	35.9
Qwen2.5-1.5B-Instruct										
Prompting	ReAct	17.4	20.5	15.7	6.2	7.7	2.0	12.8	40.1	11.3
Prompting	Reflexion	35.3	22.2	21.7	13.6	19.4	3.7	21.8	55.8	21.9
RL Training	RLOO	$88.3_{\pm 3.0}$	$52.8_{\pm 8.6}$	$71.0_{\pm 5.9}$	$62.8_{\pm 8.7}$	$66.4_{\pm 5.5}$	$56.9_{\pm 4.7}$	$69.7_{\pm 2.5}$	$73.9_{\pm 5.6}$	$52.1_{\pm 6.7}$
RL Training	EMPG	85.5	33.5	78.9	76.2	74.7	**89.1**	73.7	80.4	60.8
RL Training	$\text{GiGPO}_{\text{w/ std}}$	$94.4_{\pm 5.9}$	$67.5_{\pm 4.6}$	$94.8_{\pm 3.8}$	$94.4_{\pm 7.8}$	$79.8_{\pm 4.7}$	$76.4_{\pm 5.4}$	$86.7_{\pm 1.7}$	$83.1_{\pm 1.6}$	$65.0_{\pm 3.2}$
RL Training	$\text{GiGPO}_{\text{w/o std}}$	$\mathbf{96.0}_{\pm 1.4}$	$76.5_{\pm 3.9}$	$91.8_{\pm 5.5}$	$91.3_{\pm 6.3}$	$71.7_{\pm 8.4}$	$\mathbf{79.5}_{\pm 7.7}$	$86.1_{\pm 4.7}$	$\mathbf{83.5}_{\pm 1.8}$	$\mathbf{67.4}_{\pm 4.5}$
RL Training	PPO (with critic)	$64.8_{\pm 3.5}$	$40.5_{\pm 6.9}$	$57.1_{\pm 4.9}$	$60.6_{\pm 6.6}$	$46.4_{\pm 4.0}$	$47.4_{\pm 1.9}$	$54.4_{\pm 3.1}$	$73.8_{\pm 3.0}$	$51.5_{\pm 2.9}$
RL Training	GRPO	$85.3_{\pm 1.5}$	$53.7_{\pm 8.0}$	$84.5_{\pm 6.8}$	$78.2_{\pm 7.9}$	$59.7_{\pm 5.0}$	$53.5_{\pm 5.6}$	$72.8_{\pm 3.6}$	$75.8_{\pm 3.5}$	$56.8_{\pm 3.8}$
RL Training	**SAPO**	$92.0_{\pm 2.9}$	$\mathbf{76.9}_{\pm 6.3}$	$\mathbf{100.0}_{\pm 0.0}$	$\mathbf{100.0}_{\pm 0.0}$	$\mathbf{82.8}_{\pm 4.7}$	$82.4_{\pm 5.0}$	$\mathbf{90.1}_{\pm 2.3}$	$82.2_{1.4}$	$63.7_{1.6}$
Qwen2.5-7B-Instruct										
Prompting	ReAct	48.5	35.4	34.3	13.2	18.2	17.6	31.2	46.2	19.5
Prompting	Reflexion	62.0	41.6	44.9	30.9	36.3	23.8	42.7	58.1	28.8
RL Training	RLOO	$87.6_{\pm 4.3}$	$78.2_{\pm 8.3}$	$87.3_{\pm 5.8}$	$81.3_{\pm 7.6}$	$71.9_{\pm 5.2}$	$48.9_{\pm 8.4}$	$75.5_{\pm 4.6}$	$80.3_{\pm 3.2}$	$65.7_{\pm 4.0}$
RL Training	EMPG	92.9	75.2	74.8	86.3	73.7	65.3	78.5	81.0	69.3
RL Training	$\text{GiGPO}_{\text{w/ std}}$	$97.7_{\pm 1.6}$	$82.7_{\pm 7.9}$	$98.8_{\pm 1.6}$	$83.7_{\pm 7.2}$	$\mathbf{89.3}_{\pm 8.2}$	$79.2_{\pm 6.6}$	$90.8_{\pm 1.3}$	$84.4_{\pm 2.9}$	$72.8_{\pm 3.2}$
RL Training	$\text{GiGPO}_{\text{w/o std}}$	$91.8_{\pm 5.4}$	$\mathbf{88.6}_{\pm 6.3}$	$95.9_{\pm 3.2}$	$90.2_{\pm 2.6}$	$86.5_{\pm 5.5}$	$85.2_{\pm 7.5}$	$90.2_{\pm 2.3}$	$86.2_{\pm 2.6}$	$75.2_{\pm 3.8}$
RL Training	PPO (with critic)	$92.3_{\pm 4.0}$	$64.0_{\pm 8.4}$	$92.5_{\pm 2.4}$	$89.5_{\pm 7.0}$	$80.3_{\pm 2.0}$	$68.8_{\pm 8.3}$	$80.4_{\pm 2.7}$	$81.4_{\pm 3.1}$	$68.7_{\pm 5.1}$
RL Training	GRPO	$90.8_{\pm 5.1}$	$66.1_{\pm 6.7}$	$89.3_{\pm 5.4}$	$74.7_{\pm 6.9}$	$72.5_{\pm 5.4}$	$64.7_{\pm 7.3}$	$77.6_{\pm 5.2}$	$79.3_{\pm 2.8}$	$66.1_{\pm 3.7}$
RL Training	**SAPO**	$\mathbf{99.0}_{\pm 1.4}$	$82.3_{\pm 2.1}$	$\mathbf{100.0}_{\pm 0.0}$	$\mathbf{97.9}_{\pm 4.7}$	$79.7_{\pm 3.9}$	$\mathbf{91.7}_{\pm 1.6}$	$\mathbf{94.0}_{\pm 1.7}$	$\mathbf{88.6}_{\pm 1.8}$	$\mathbf{82.4}_{\pm 2.0}$

Table 1: Evaluation results on ALFWorld and WebShop. For each RL training method, we report the mean and standard deviation over three random seeds. The ALFWorld contains six categories: Pick & Place (Pick), Examine in Light (Look), Clean & Place (Clean), Heat & Place (Heat), Cool & Place (Cool), and Pick Two & Place (Pick2). Most entries in this table are reported by Feng et al. [4]. Notably, the baseline $\text{GiGPO}_{\text{w/o std}}$ replaces the group-relative normalization std with one.

5 Experiments

We evaluate SAPO across a range of multi-turn environments. Specifically, our experiments aim to answer four research questions: (1) *How does SAPO perform compared with the core PPO and GRPO baselines?* (2) *How sensitive is SAPO to its components, and how much does each component*

contribute? (3) *How does SAPO compare with a comparable PPO baseline in terms of resource overhead?* (4) *Can SAPO effectively maintain stable performance over continuous training iterations over long-horizon tasks?*

5.1 Experimental Setup

Benchmarks and Baselines. We train and evaluate SAPO on two challenging multi-turn ALFWorld [20] and WebShop [27] benchmarks. ALFWorld is an embodied household environment for evaluating long-horizon textual reasoning and decision-making. In each episode, the agent receives a concrete task instruction sampled from 3,827 tasks from six categories. WebShop is an interactive web-shopping environment containing nearly 1.1 million products and 12,000 user instructions. We compare SAPO against three categories of competitive baselines: (1) Proprietary models: GPT-4o [1] and Gemini-2.5-Pro [22]; (2) training-free prompting agents: ReAct [28] and Reflexion [19]; and (3) RL-based methods: PPO [16], RLOO [2], GRPO [17], EMPG [24] and GiGPO[4].

Implementation details. We employ Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct as the base models for training experiments. To ensure fair comparisons, the hyperparameter settings of SAPO follow the existing RL framework [4] in each benchmark. Specifically, we use a learning rate of 1×10^{-6}, and a KL-penalty coefficient of 0.01. The maximum number of interaction turns is set to 50 for ALFWorld and 15 for WebShop, and we train for 150 steps overall. All our experiments were run on 4 NVIDIA H200s and 8 NVIDIA A40s. The technical supplement provides additional implementation details.

5.2 Experiment Results

As shown in Table 1, across ALFWorld and WebShop with Qwen2.5-1.5B/7B, SAPO consistently outperforms standard PPO and GRPO at both model scales, while matching or exceeding recent improved variants overall, with particularly clear gains at Qwen2.5-7B. For example, with Qwen2.5-1.5B, SAPO achieves 90.1% aggregate success on ALFWorld, improving over PPO and GRPO by 35.7 and 17.3 percentage points, respectively, and reaches perfect performance on the Clean and Heat categories. On WebShop, it improves PPO by 8.4 points in score and 12.2 points in success rate. The highlighted advantage persists at 7B, i.e., SAPO obtains 94.0% ALFWorld success and an 88.6 WebShop score with 82.4% success, outperforming both PPO and GRPO as well as the recent variants on all three aggregate metrics. Although SAPO is not uniformly best on every individual category, its aggregate performance is consistently best.

Overall, the results indicate that SAPO's gains are not confined to a specific model capacity or environment. By integrating value estimation and policy optimization within a single-rollout au-

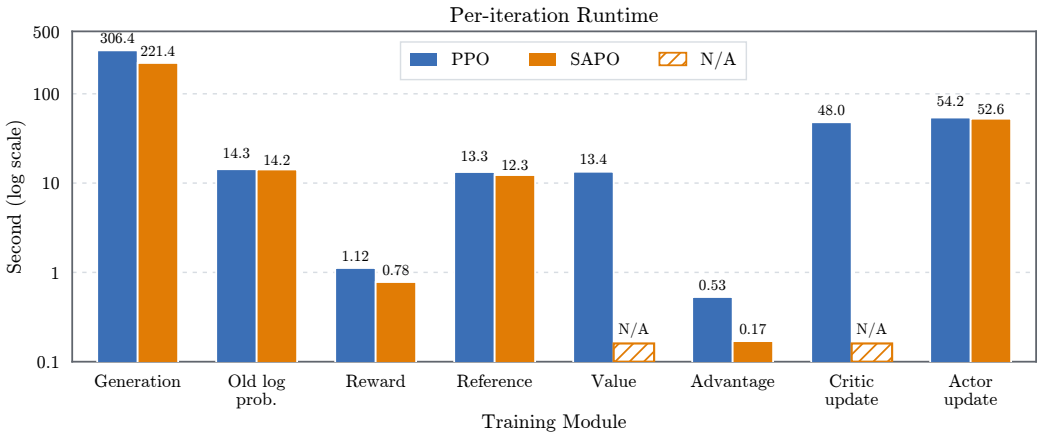

Figure 2: Per-iteration runtime breakdown of PPO and SAPO on ALFWorld using Qwen2.5-1.5B. The vertical axis is shown on a logarithmic scale to accommodate the large variation in module runtimes. *N/A* indicates that SAPO does not require the corresponding value-model or critic-update module. Overall, SAPO reduces the measured per-iteration runtime from 451.2 s to 301.4 s, corresponding to a 33.2% reduction compared with PPO.

toregressive process, SAPO retains effective temporal credit assignment without a separate critic or group-based sampling. Its strong performance using sparse outcome rewards further suggests that the architectural alignment between autoregressive modeling and actor–critic learning can reduce dependence on task-specific dense reward engineering.

5.3 Per-iteration Runtime Comparison

Figure 2 compares the per-iteration runtime breakdown of PPO and SAPO on ALFWorld using Qwen2.5-1.5B. SAPO reduces the total measured runtime from 451.2 s to 301.4 s, yielding a 33.2% reduction over PPO. The largest absolute saving comes from trajectory generation, whose runtime decreases from 306.4 s to 221.4 s. More importantly, SAPO eliminates the separate value inference and critic optimization stages, which together account for 61.4 s per iteration in PPO. In contrast, the remaining actor-side costs are comparable: old-policy log-probability computation takes 14.3 s versus 14.2 s, reference-model evaluation takes 13.3 s versus 12.3 s, and actor updates take 54.2 s versus 52.6 s for PPO and SAPO, respectively. These results indicate that integrating value estimation into the shared autoregressive model introduces little additional actor-side overhead. Overall, SAPO improves runtime efficiency structurally by removing the independent critic pathway while retaining explicit value learning and temporal credit assignment.

6 Conclusion

We introduced SAPO, a single-rollout autoregressive actor–critic framework for RL of LLM agents. SAPO exploits causal boundaries within one language model to represent the policy, state value, and action value without a separate critic network or group-relative sampling. The unified objective jointly optimizes policy and value learning through a shared backbone while preserving their distinct conditioning and supervision. Experiments on ALFWorld and WebShop with Qwen2.5-1.5B and 7B demonstrate stable training and task performance, outperforming PPO and GRPO while using only one rollout per task. SAPO also removes the memory cost of a policy-scale critic and reduces measured per-iteration runtime by 33.2% relative to PPO. These results show that explicit value learning need not require duplicated models or costly sampling, offering a path toward efficient long-horizon agentic RL.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report, 2023.

[2] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[3] Yunho Choi, Jongwon Lim, Woojin Ahn, Minjae Oh, Jeonghoon Shim, and Yohan Jo. Your language model is its own critic: Reinforcement learning with value estimation from actor's internal states, 2026.

[4] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training, 2026.

[5] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025.

[6] Zhenyu Hou, Yujiang Li, Jie Tang, and Yuxiao Dong. Single-rollout asynchronous optimization for agentic reinforcement learning, 2026.

[7] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-Reasoner-Zero: An open source approach to scaling up reinforcement learning on the

base model. In *Advances in Neural Information Processing Systems*, volume 38, pages 162239–162262. Curran Associates, Inc., 2025.

[8] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning, 2025.

[9] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Unlocking RL potential for LLM reasoning through refined credit assignment, 2024.

[10] Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, et al. Websailor: Navigating super-human reasoning for web agent, 2025.

[11] Xuefeng Li, Haoyang Zou, and Pengfei Liu. ToRL: Scaling tool-integrated reinforcement learning, 2025.

[12] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 29128–29163. PMLR, 2024.

[13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022.

[14] Jiangweizhi Peng, Yuanxin Liu, Ruida Zhou, Charles Fleming, Zhaoran Wang, Alfredo Garcia, and Mingyi Hong. HiPER: Hierarchical plan–execute reinforcement learning for multi-turn LLM agents. In *Forty-third International Conference on Machine Learning*, 2026.

[15] Michael Santacroce, Yadong Lu, Han Yu, Yuanzhi Li, and Yelong Shen. Efficient RLHF: Reducing the memory usage of PPO, 2023.

[16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[17] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024.

[18] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

[20] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning, 2020.

[21] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021. Curran Associates, Inc., 2020.

[22] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models, 2023.

[23] Guoqing Wang, Sunhao Dai, Guangze Ye, Zeyu Gan, Wei Yao, Yong Deng, Xiaofeng Wu, and Zhenzhe Ying. Information gain-based policy optimization: A simple and effective approach for multi-turn search agents. In *The Fourteenth International Conference on Learning Representations*, 2026.

[24] Jiawei Wang, Jiacai Liu, Yuqian Fu, Yingru Li, Xintao Wang, Yuan Lin, Yu Yue, Lin Zhang, Yang Wang, and WANG KE. Harnessing uncertainty: Entropy-modulated policy gradients for long-horizon LLM agents. In *Forty-third International Conference on Machine Learning*, 2026.

[25] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning, 2025.

[26] Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Jiaqi Liu, Honglin Guo, Yajie Yang, Rui Zheng, Junjie Ye, Jiazheng Zhang, et al. AgentGym-RL: An open-source framework to train LLM agents for long-horizon decision making via multi-turn reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026.

[27] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2022.

[28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022.

[29] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025.

[30] Yufeng Yuan, Yu Yue, Ruofei Zhu, Tiantian Fan, and Lin Yan. What's behind PPO's collapse in long-CoT? value optimization holds the secret, 2025.

[31] Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, Tiantian Fan, Zhengyin Du, Xiangpeng Wei, Xiangyu Yu, Gaohong Liu, Juncai Liu, Lingjun Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Ru Zhang, Xin Liu, Mingxuan Wang, Yonghui Wu, and Lin Yan. VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks, 2025.

[32] Kaichen Zhang, Yuzhong Hong, Junwei Bao, Hongfei Jiang, Yang Song, Hong Dingqian, and Hui Xiong. GVPO: Group variance policy optimization for large language model post-training, 2025.

[33] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization, 2025.

[34] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 414–431, 2025.

[35] Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. ARChER: Training language model agents via hierarchical multi-turn reinforcement learning, 2024.